llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5 · claude-haiku-4-5-20251001 · FASTER

−0 / +6 lines of 1967 · 6 parallel region(s) added · 10 of 25 candidate(s) kept · ANNOTATION · restructuring level 0

BEFORE

```
656      Real_t *dvdz = Allocate<Real_t>(numElem8) ;
657      Real_t *x8n  = Allocate<Real_t>(numElem8) ;
658      Real_t *y8n  = Allocate<Real_t>(numElem8) ;
659      Real_t *z8n  = Allocate<Real_t>(numElem8) ;
660
661      for (Index_t i=0 ; i<numElem ; ++i){
662         Real_t  x1[8],  y1[8],  z1[8] ;
663         Real_t pfx[8], pfy[8], pfz[8] ;
664
665         Index_t* elemToNode = domain.nodelist(i);
     …
749
750   static inline
751   void CalcAccelerationForNodes(Domain &domain, Index_t numNode)
752   {
753
754      for (Index_t i = 0; i < numNode; ++i) {
755         domain.xdd(i) = domain.fx(i) / domain.nodalMass(i);
756         domain.ydd(i) = domain.fy(i) / domain.nodalMass(i);
757         domain.zdd(i) = domain.fz(i) / domain.nodalMass(i);
758      }
     …
785   static inline
786   void CalcVelocityForNodes(Domain &domain, const Real_t dt, const Real_t u_cut,
787                             Index_t numNode)
788   {
789
790      for ( Index_t i = 0 ; i < numNode ; ++i )
791      {
792        Real_t xdtmp, ydtmp, zdtmp ;
793
794        xdtmp = domain.xd(i) + domain.xdd(i) * dt ;
     …
806   }
807
808   static inline
809   void CalcPositionForNodes(Domain &domain, const Real_t dt, Index_t numNode)
810   {
811      for ( Index_t i = 0 ; i < numNode ; ++i )
812      {
813        domain.x(i) += domain.xd(i) * dt ;
814        domain.y(i) += domain.yd(i) * dt ;
815        domain.z(i) += domain.zd(i) * dt ;
     …
1061
1062  void CalcKinematicsForElems( Domain &domain,
1063                               Real_t deltaTime, Index_t numElem )
1064  {
1065
1066    for( Index_t k=0 ; k<numElem ; ++k )
1067    {
1068      Real_t B[3][8] ;
1069      Real_t D[6] ;
1070      Real_t x_local[8] ;
     …
1150  static inline
1151  void CalcMonotonicQGradientsForElems(Domain& domain)
1152  {
1153     Index_t numElem = domain.numElem();
1154
1155     for (Index_t i = 0 ; i < numElem ; ++i ) {
1156        const Real_t ptiny = Real_t(1.e-36) ;
1157        Real_t ax,ay,az ;
1158        Real_t dxv,dyv,dzv ;
1159
```

AFTER

```
656      Real_t *dvdz = Allocate<Real_t>(numElem8) ;
657      Real_t *x8n  = Allocate<Real_t>(numElem8) ;
658      Real_t *y8n  = Allocate<Real_t>(numElem8) ;
659      Real_t *z8n  = Allocate<Real_t>(numElem8) ;
660
661      #pragma omp parallel for firstprivate(numElem) shared(x8n,dvdy,z8n,dvdx,dvd
662      for (Index_t i=0 ; i<numElem ; ++i){
663         Real_t  x1[8],  y1[8],  z1[8] ;
664         Real_t pfx[8], pfy[8], pfz[8] ;
665
666         Index_t* elemToNode = domain.nodelist(i);
     …
750
751   static inline
752   void CalcAccelerationForNodes(Domain &domain, Index_t numNode)
753   {
754
755      #pragma omp parallel for firstprivate(numNode) shared(domain)
756      for (Index_t i = 0; i < numNode; ++i) {
757         domain.xdd(i) = domain.fx(i) / domain.nodalMass(i);
758         domain.ydd(i) = domain.fy(i) / domain.nodalMass(i);
759         domain.zdd(i) = domain.fz(i) / domain.nodalMass(i);
760      }
     …
787   static inline
788   void CalcVelocityForNodes(Domain &domain, const Real_t dt, const Real_t u_cut,
789                             Index_t numNode)
790   {
791
792      #pragma omp parallel for firstprivate(dt,u_cut,numNode) shared(domain)
793      for ( Index_t i = 0 ; i < numNode ; ++i )
794      {
795        Real_t xdtmp, ydtmp, zdtmp ;
796
797        xdtmp = domain.xd(i) + domain.xdd(i) * dt ;
     …
809   }
810
811   static inline
812   void CalcPositionForNodes(Domain &domain, const Real_t dt, Index_t numNode)
813   {
814      #pragma omp parallel for firstprivate(dt,numNode) shared(domain)
815      for ( Index_t i = 0 ; i < numNode ; ++i )
816      {
817        domain.x(i) += domain.xd(i) * dt ;
818        domain.y(i) += domain.yd(i) * dt ;
819        domain.z(i) += domain.zd(i) * dt ;
     …
1065
1066  void CalcKinematicsForElems( Domain &domain,
1067                               Real_t deltaTime, Index_t numElem )
1068  {
1069
1070    #pragma omp parallel for firstprivate(deltaTime,numElem) shared(domain)
1071    for( Index_t k=0 ; k<numElem ; ++k )
1072    {
1073      Real_t B[3][8] ;
1074      Real_t D[6] ;
1075      Real_t x_local[8] ;
     …
1155  static inline
1156  void CalcMonotonicQGradientsForElems(Domain& domain)
1157  {
1158     Index_t numElem = domain.numElem();
1159
1160     #pragma omp parallel for firstprivate(numElem) shared(domain)
1161     for (Index_t i = 0 ; i < numElem ; ++i ) {
1162        const Real_t ptiny = Real_t(1.e-36) ;
1163        Real_t ax,ay,az ;
1164        Real_t dxv,dyv,dzv ;
1165
```

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is